

KOPS + Prometheus + Grafana + Alertmanager Setup

1. Create KOPS Cluster

Create a KOPS cluster with one master node and two worker nodes.

```
Master: m7i
Workers: c7i (2 nodes)
```

2. Install Helm

Install Helm on the environment from which you manage the Kubernetes cluster.

3. Install Prometheus and Grafana

```
helm install prometheus oci://ghcr.io/prometheus-community/charts/kube-prometheus-stack
```

4. Check Pods

```
kubectl get pods
```

5. Get Grafana Admin Password

```
kubectl --namespace default get secrets prometheus-grafana -o jsonpath="{.data.admin-password}" | base64 -d ; echo
```

6. Check Services

```
kubectl get svc
```

7. Expose Grafana and Prometheus

```
kubectl patch svc prometheus-grafana -p '{"spec":{"type":"LoadBalancer"}}'
kubectl patch svc prometheus-kube-prometheus-prometheus -p '{"spec":{"type":"LoadBalancer"}}'
```

8. Access the Dashboards

Use the external LoadBalancer endpoints from `kubectl get svc` to access the Grafana and Prometheus dashboards.

9. Create Alertmanager Configuration

```
cd alert-config/
vim alert-config.yaml
```

alert-config.yaml

Use a placeholder for the Slack webhook. Do not commit a real webhook URL to Git.

```
apiVersion: v1
kind: Secret
metadata:
  name: slack-webhook
  labels:
    release: prometheus
type: Opaque
stringData:
  slack_webhook_url: "https://hooks.slack.com/services/REPLACE/THIS/WITH/YOUR/WEBHOOK"
---
apiVersion: monitoring.coreos.com/v1alpha1
kind: AlertmanagerConfig
metadata:
  name: spring-alertmanager-config
  labels:
```

```

    release: prometheus
spec:
  route:
    groupBy: ["alertname"]
    groupWait: 30s
    groupInterval: 5m
    repeatInterval: 1h
    continue: true
    receiver: "slack-notification"
    routes:
      - receiver: "slack-notification"
        continue: true
  receivers:
    - name: "slack-notification"
      slackConfigs:
        - channel: "new-channel"
          apiURL:
            name: slack-webhook
            key: slack_webhook_url
          sendResolved: true
          title: "{{ range .Alerts }}{{ .Annotations.summary }}\n{{ end }}"
          text: "{{ range .Alerts }}{{ .Annotations.description }}\n{{ end }}"

```

10. Apply the Alertmanager Configuration

```
kubectl apply -f alert-config.yaml
```

11. Create a Test Pod

```

kubectl run pod-1 --image=nginx
kubectl exec -it pod-1 -- bash

```

Security Note

Never commit a real Slack Incoming Webhook URL to a public repository. Use a placeholder in practice files and inject the real webhook securely at deployment time.